

Domande per l'Orale LP

Modulo I/II/III

Contents

1	Modulo 1 – Gorrieri	2
1.1	Capitolo 1 – Macchine astratte, interpreti, compilatori	2
1.2	Capitolo 2 – Descrivere un linguaggio di programmazione	5
1.3	Capitolo 3 – Analisi lessicale, linguaggi regolari	11
1.4	Capitolo 4 – Analisi sintattica, linguaggi liberi	22
1.5	Capitolo 5 – Fondamenti	44
2	Modulo 2 – Gabbrielli	48
3	Modulo 3 – Giallorenzo	51

1 Modulo 1 – Gorrieri

1.1 Capitolo 1 – Macchine astratte, interpreti, compilatori

Domanda 1

Che cosa è una macchina astratta? E in cosa si differenzia da una macchina fisica?

E' un insieme di algoritmi e strutture dati che dato un programma scritto in un linguaggio relativo alla macchina astratta, quest'ultima lo esegue.

Una macchina fisica e' un insieme di circuiti logici composti da circuiti elettrici che allo stesso modo della macchina astratta dato un linguaggio specifico (conforme all'IS), la macchina fisica lo esegue.

Domanda 2

Che cos'è un interprete? In cosa consiste il ciclo fetch-decode-execute?

Un interprete e' un programma che viene eseguito su un calcolatore che serve per eseguire in realtime un programma, senza prima doverlo compilare. Serve per poter eseguire un linguaggio $\mathcal{L}_A$ su un calcolatore che nativamente esegue il linguaggio $\mathcal{L}$. L'interprete esegue il codice leggendo una istruzione alla volta.

FDE e' lo schema della CPU nel quale prende, decodifica ed esegue una istruzione. Questo e' fortemente collegato alla pipeline della CPU.

Domanda 3

Cos'è il linguaggio macchina?

E' linguaggio relativo all'interprete $I_{\mathcal{L}}$ per la macchina $M_{\mathcal{L}}$.

Domanda 4

Possono esistere macchine diverse con lo stesso linguaggio macchina?

Certamente, infatti possiamo avere macchine astratte o fisiche che implementano o adattano ISA a seconda del linguaggio.

Domanda 5

In quali modi è possibile implementare una macchina astratta? Elencare vantaggi e svantaggi delle varie tecniche.

In 3 modi:

- 1. A livello software: piu' lento ma il piu' flessibile*
- 2. A livello firmware: abbastanza veloce, abbastanza flessibile*
- 3. A livello hardware: piu' veloce ma meno flessibile*

Questo perche' ogni aumento di livello porta un overhead maggiore, quindi abbiamo un tradeoff fra performance e velocita' di implementazione da parte del programmatore.

Domanda 6

Che cos'e un compilatore?

Un compilatore e' un programma che traduce programma scritti in $\mathcal{L}$ in altri linguaggi in $\mathcal{L}_0$:

$$C_{\mathcal{L}, \mathcal{L}_0} : P^{\mathcal{L}} \rightarrow P^{\mathcal{L}_0}$$

Domanda 7

Relativamente alla tecnica d'implementazione software, descrivere la tecnica d'implementazione interpretativa pura e quella compilativa pura.

Il compilatore e' un traduttore da un linguaggio $\mathcal{L}$ a un altro linguaggio $\mathcal{L}_0$, questo traduce da inizio a fine e ritorna un nuovo programma in $\mathcal{L}_0$. Mentre un interprete e' una macchina astratta dove esegue riga per riga alla volta una operazione.

Domanda 8

Quando un interprete si puo dire corretto? Quando un compilatore si puo dire corretto?

Un interprete o compilatore si puo' dire corretto quando l'output restituito dal programma/esecuzione combacia di relazione dominio $\Rightarrow$ codominio con quello nativo.

Domanda 9

Confrontare l'implementazione di una macchina astratta su una macchina ospite per mezzo di un interprete o di un compilatore.

Data una macchina fisica M con linguaggio $\mathcal{L}$ possiamo eseguire programmi scritti in $\mathcal{L}_0$ grazie a un interprete o compilatore t.c.:

$$I_{\mathcal{L}}^{\mathcal{L}_0}$$

L'interpretazione esegue ogni istruzione di $\mathcal{L}_0$, istruzione per istruzione, sulla macchina astratta che gira in M . Mentre il compilatore traduce a priorio e poi se si vuole si esegue il nuovo programma tradotto da $\mathcal{L}_0$ in linguaggio nativo $\mathcal{L}$.

Domanda 10

Come vengono implementate nella realta le macchine astratte? Che cos'e la macchina intermedia?

In una macchina fisica per implementare il linguaggio $\mathcal{L}_0$ possiamo tradurlo prima in un IR/Bytecode $\mathcal{L}_1$ che verra' eseguito su una macchina astratta (intermediaria) fra $\mathcal{L}$ e $\mathcal{L}_0$:

$$\mathcal{L}_0 \rightarrow \mathcal{L}_1 \rightarrow \mathcal{L}$$

Domanda 11

Quando si dice che una implementazione e di tipo interpretativo e quando di tipo compilativo? Fare esempi di linguaggi la cui implementazione e di un tipo o dell'altro.

E' interpretativo quando utilizziamo un interprete, mentre e' compilativo quando utilizziamo un compilatore. Per esempio python e' interpretativo, mentre C e' compilativo.

Domanda 12

Cos'e la Just-in-Time compilation?

E' una via di mezzo fra interpretazione e compilazione. Cerca di abbattere i costi di rallentamento dell'interpretazione avendo un codice compilato, pero' non dovendo compilare tutto a priori ma solo a chiamata quando serve in esecuzione.

Domanda 13

L'interprete e il compilatore si possono sempre realizzare?

Il compilatore puo' essere sempre realizzato anche perche' puo' girare su una macchina esterna rispetto a i linguaggi in input-output. Mentre l'interprete ha un costo di implementazione maggiore, quindi non e' detto che sia supportato. Bisogna poi tenere conto anche del livello di espressionalita' dei linguaggi. Un linguaggio con potere maggiore non puo' essere convertito in uno di potere minore.

Domanda 14

Che cosa e l'implementazione via kernel?

Si divide il linguaggio $\mathcal{L}_0$ in due parti:

- *Primitive: operazioni essenziali*
- *Derivate: operazioni che poi vengono ridotti (tradotte) nelle primitive*

Il vantaggio e' che dobbiamo implementare solo poche operazione, ovvero le primitive, il resto e' solo traduzione statica.

Domanda 15

Quando si parla di bootstrapping?

E' quando per implementare il linguaggio $\mathcal{L}$ scriviamo un compilatore o un interprete in $\mathcal{L}$.

1.2 Capitolo 2 – Descrivere un linguaggio di programmazione

Domanda 16

Quali sono i livelli di descrizione di un linguaggio?

1. *Sintassi: parole (token)*
2. *Semantica: costruzione delle frasi (es. assegnazione tipo sbagliato)*
3. *Pragmatica: come lo implementiamo*

Domanda 17

Sintassi: qual e l'aspetto lessicale e quale quello grammaticale di un linguaggio?

Il lessico sono le parole (token) consentite, mentre la grammatica e' l'ordine delle parole.

Domanda 18

Cos'e un alfabeto? Cos'e una parola o stringa? Cos'e A^* ? Tale insieme e enumerabile?

Un alfabeto A e' un insieme finito di simboli non vuoti. Una parola (o stringa) e' una sequenza di lettere dell'alfabeto. Definiamo A^ l'insieme di tutte le parole finite (inclusa la vuota) di A :*

$$A^* = \bigcup_{i \geq 0} A^i$$

Se A e' finito allora possiamo ordinare per lunghezza tutte le parole, ottenendo una sequenza numerabili di numeri naturali.

Domanda 19

Definizione di potenza di una stringa. Definizione di potenza di un linguaggio. Definizione di chiusura/iterazione (o stella di Kleene) di un linguaggio.

La n -esima potenza della stringa w e' definita come:

$$w^n = \begin{cases} \epsilon & n = 0 \\ ww^{n-1} & n > 0 \end{cases}$$

Dato un alfabeto A e L l'insieme di stringhe (linguaggio), definiamo la stella di Kleene:

$$L^* = \bigcup_{i \geq 0} L^i$$

con

- $L^0 = \{\epsilon\}$
- $L^{n+1} = LL^n$

Domanda 20

Definizione di grammatica libera da contesto. Come si deriva una stringa? Qual e il linguaggio generato da una grammatica libera?

Una CFG (PDA) e' una quadra

$$CFG = (V, T, P, S)$$

- V : variabili (non terminali)
- T : terminali
- P : produzioni ($A \rightarrow a|b$)
- S : simboli di start

Data una CFG deriviamo w da v se esiste una transizione riflessiva:

$$v \rightarrow w_i \rightarrow w$$

ovvero

$$\frac{}{v \rightarrow^* w_i} \quad \frac{v \rightarrow^* w}{v \rightarrow^* w_i w_i \rightarrow w}$$

Il linguaggio generato da una CFG è l'insieme di tutte le stringhe di terminali derivabili da S (simboli di start)

Domanda 21

Cos'è un albero di derivazione? Cos'è una derivazione canonica sinistra/destra? Esiste una corrispondenza biunivoca tra alberi di derivazioni e derivazioni canoniche?

Un albero di derivazione e' una rappresentazione grafica di una derivazione in una CFG.

Per calcolare l'albero di derivazione dobbiamo prima espandere i non terminali, quindi dobbiamo decidere se partire da destra o sinistra (sono le derivazioni canoniche). Quindi l'albero di derivazione e' equivalente alle espansioni canoniche (sono biunivoche).

Domanda 22

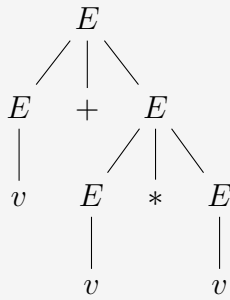
Quando una grammatica è ambigua? (fare un esempio) Quando un linguaggio è ambiguo? (fare un esempio)

Una grammatica e' ambigua quando data una stringa esistono più di un albero di derivazione **distinto**:

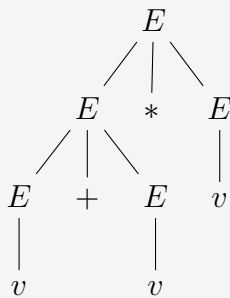
$$E \rightarrow E + E | E * E | (E) | v$$

ESEMPIO: $v + v * v$

1. $(v + v) * v$



2. $v + (v * v)$



Un linguaggio e' ambiguo quando possiamo formularlo con una grammatica ambigua.
ESEMPIO: $L = \text{espressione su } v$.

Domanda 23

E' possibile rimuovere l'ambiguita' dalla grammatica delle espressioni aritmetiche? Come?

Si e' possibile introducendo priorit  fra gli operatori aritmetici.

$$\begin{aligned}
 E &\rightarrow E + T \mid T \\
 T &\rightarrow T * F \mid F \\
 F &\rightarrow (E) \mid v
 \end{aligned}$$

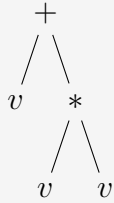
Ovvero essendo che il $*$ sta a un livello piu' profondo del $+$ allora ha una precedenza maggiore.

Domanda 24

Cos'e' l'albero di sintassi astratta? Che differenza c'e' tra sintassi concreta e sintassi astratta? Cos'e' lo zucchero sintattico?

L'AST e' l'albero ripulito dai non terminali e terminali dell'albero di derivazione e anche dalla sintassi non essenziali. E' una rappresentazione logica della struttura essenziali del linguaggio di programmazione.

*ESEMPIO: $v+v*v$*



Lo zucchero sintattico sono tutti quei costrutti di un LP che servono a migliorare la leggibilità del programma rispetto alla implementazione (es. i `for` sono sotto `while`).

Domanda 25

Fare esempi di vincoli sintattici contestuali. Possono essere catturati attraverso grammatiche libere?

I vincoli sintattici contestuali sono regole che non dipendono solo dalla forma della stringa, ma dal contesto in cui compaiono simboli o costrutti. (ovvero tipo uso di variabili non prima dichiarate). Queste non possono essere rilevate da CFG.

Domanda 26

Cosa s'intende per semantica statica? E per semantica dinamica?

La semantica statica copre tutti i controlli statici sul linguaggio (es. conflitto di tipi, var. non dichiarata), mentre la semantica dinamica scrive il comportamento del programma durante l'esecuzione (es. if-else, operazioni aritmetiche).

Domanda 27

Elencare le varie fasi in cui si articola un compilatore. Descrivere in dettaglio ogni singola fase.

1. *Lexer: genera il sorgente in una lista di token*
2. *Parser: Genera AST, verifica regole grammaticali (ordine parole)*
3. *Analisi semantica statica: controllo su conflitti di tipi, dichiarazione di variabili*
4. *Generazione IR*
5. *Ottimizzazione (facoltativa)*
6. *Generazione del codice per la architettura*

Domanda 28

A chi serve definire la semantica dinamica di un linguaggio e perché?

La semantica dinamica è la base teorica dell'esecuzione degli interpreti o del derivato di un compilatore. Serve all'implementatore/sviluppatore.

Domanda 29

Quali tecniche si usano per dare semantica dinamica ad un linguaggio di programmazione?

1. *Semantica operativa: descrive l'esecuzione come una sequenza di passi su una macchina astratta*
2. *Semantica denotazionale: associa a ogni costrutto del linguaggio un oggetto matematico*
3. *Semantica assiomatica: descrive il comportamento tramite logica e predicati (Hoare logic) (tipo $\text{precondizione} \rightarrow \text{comando} \rightarrow \text{postcondizione}$)*

Domanda 30

Imparare le regole di semantica operativa SOS per il semplice linguaggio presentato a lezione. Regole di valutazione: interna vs esterna, sinistra vs destra vs parallela.

La valutazione interna descrive ogni passaggio intermedio, mentre la esterna solo il risultato finale.

La valutazione destra, sinistra servono per indicare la priorità di valutazione in caso di due argomenti. Parallela invece li valuta contemporaneamente.

$$\begin{array}{c}
\langle V, \sigma \rangle \rightarrow_e \langle \underbrace{\sigma(V), \sigma}_{\text{stato terminale}} \rangle \text{ VAR} \\
\\
\frac{\langle e_0, \sigma \rangle \rightarrow_e \langle e'_0, \sigma' \rangle}{\langle e_0 + e_1, \sigma \rangle \rightarrow_e \langle e'_0 + e_1, \sigma' \rangle} \text{ SUM1} \qquad \frac{\langle e_1, \sigma \rangle \rightarrow_e \langle e'_1, \sigma' \rangle}{\langle m + e_1, \sigma \rangle \rightarrow_e \langle m + e'_1, \sigma' \rangle} \text{ SUM2} \\
\\
\frac{}{\langle m + m', \sigma \rangle \rightarrow_e \langle P, \sigma \rangle} \text{ SUM3} \quad \text{se } P = m + m' \\
\\
\frac{\langle e_0, \sigma \rangle \rightarrow_e \langle e'_0, \sigma' \rangle}{\langle e_0 - e_1, \sigma \rangle \rightarrow_e \langle e'_0 - e_1, \sigma' \rangle} \text{ SUB1} \qquad \frac{\langle e_1, \sigma \rangle \rightarrow_e \langle e'_1, \sigma' \rangle}{\langle m - e_1, \sigma \rangle \rightarrow_e \langle m - e'_1, \sigma' \rangle} \text{ SUB2} \\
\\
\frac{}{\langle m - m', \sigma \rangle \rightarrow_e \langle P, \sigma \rangle} \text{ SUB3} \quad \text{se } m \geq m' \text{ e } P = m - m'
\end{array}$$

Domanda 31

Cosa s'intende per **pragmatica**? Cosa si intende per **implementazione** di un linguaggio?

Per pragmatica di un linguaggio si intende l'insieme degli aspetti legati all'uso del linguaggio da parte del programmatore e agli effetti concreti dell'esecuzione dei programmi, cioè come il linguaggio si comporta nella pratica (efficienza, gestione della memoria, effetti collaterali, modello di esecuzione, ecc.).
Con implementazione intendiamo la realizzazione concreta del linguaggio attraverso un interprete o compilatore.

1.3 Capitolo 3 – Analisi lessicale, linguaggi regolari

Domanda 32

Cosa fa l'analizzatore lessicale?

Il lexer prende il sorgente e restituisce una lista di token (identificatori, numeri, operazioni, ecc)

Domanda 33

Cos'è un token?

Un token è un identificatore che il programmatore del compilatore ha definito per specificare che quella parola (stringa) sia di quella tipologia.

Domanda 34

Cos'è un pattern? Come lo si rappresenta? Cos'è un lessema?

Un pattern è una descrizione formale che specifica quali stringhe appartengono a una certa categoria lessicale.

- *Regex*
- *NFA-DFA*

L'output del lexer è una lista di tuple (token, lessema)

Domanda 35

Definizione di espressioni regolari (sintassi) e di linguaggio associato (semantica).

Dato alfabeto A con lettere $a_i \in A$ definiamo espressioni regolari:

$$E \rightarrow \emptyset \mid \epsilon \mid a_i \mid r \cdot r \mid E \text{ or } E \mid E^*$$

Definiamo linguaggio regolare:

- $L[\emptyset] = \emptyset$
- $L[\epsilon] = \{\epsilon\}$
- $L[a_i] = \{a_i\}$, per ogni simbolo $a_i \in A$
- $L[E_1 \cdot E_2] = \{xy \mid x \in L[E_1], y \in L[E_2]\}$ (concatenazione)
- $L[E_1 \mid E_2] = L[E_1] \cup L[E_2]$ (unione / or)
- $L[E^*] = \bigcup_{n \geq 0} L[E]^n$ (stella di Kleene)

Domanda 36

Quali sono i linguaggi regolari? I linguaggi finiti sono tutti regolari? Esistono linguaggi infiniti regolari?

I linguaggi regolari sono una classe dei linguaggi formali. Sono descritti da:

- *Espressioni regolari*
- *Automi a stati finiti (DFA-NFA)*
- *CFG*

I linguaggi finiti sono tutti regolari, infatti essendo finiti possiamo definire una espressione regolare ad-hoc:

$$L = a, bb, ccc \Rightarrow a \mid bb \mid ccc$$

Esistono pero' linguaggi infiniti (solo alcuni non tutti!) che sono regolari:

$$L = \{\epsilon, a, aa, aaa, \dots\} \Rightarrow aA \mid \epsilon$$

Domanda 37

Definizione di equivalenza tra espressioni regolari. Elencare alcune leggi di equivalenza.

Una definizione regolare su alfabeto A è costituita da una lista di definizioni regolari

$$d_1 := E_1$$

$\vdots$

$$d_n := E_n$$

definiamo d_i simboli nuovi da utilizzare per ampliare l'alfabeto $(A \cup d_i)$.

ESEMPIO:

$$\text{identifier} := \text{letter}(\text{letter}|\text{digit})^*$$

$$\text{letter} := [a - zA - Z]$$

$$\text{digit} := [0 - 9]$$

Dato E_1 e E_2 come espressioni regolari, definiamo equivalenti L_1 e L_2 se

$$L_1[E_1] = L_2[E_2]$$

Domanda 38

Cos'è una definizione regolare e a cosa serve?

E' una lista di espressioni regolari, e servono per ampliare l'alfabeto di un linguaggio attraverso lettere + espressioni.

Domanda 39

Definizione di NFA (automa finito nondeterministico). Discutere cosa si intende per nondeterminismo. Mettere in relazione la definizione formale con la rappresentazione grafica come diagramma di transizioni.

Un NFA e' una quintupla $\{\Sigma, Q, \delta, q_0, F\}$ dove:

- Σ : input
- Q : stati totali
- δ : funzione di transizione

- $q_0 \subseteq Q$: stati iniziali
- $F \subseteq Q$: stati finali

Con nondeterminismo si intende quando esiste almeno uno stato q , che ha input a t.c.

$$\delta(q, a) = q'$$

$$\delta(q, a) = q''$$

Ovvero se esiste almeno uno stato che puo' prendere due alternative possibili ($|\delta(q, a)| \geq 1$).

Possiamo disegnare un NFA come un grafo nel quale:

- Lo stato iniziale e' q_0
- Ogni stato $q \in Q$ corrisponde a un nodo (cerchio) del grafo.
- Gli stati finali F sono rappresentati con doppio cerchio.
- Ogni transizione $q' \in \delta(q, a)$ è rappresentata da un arco (freccia) orientato da q a q' etichettato con il simbolo a .

Domanda 40

Come si definisce il linguaggio accettato da un NFA? Definizione di equivalenza tra NFA.

Il linguaggio accettato per stato finali di un NFA si definisce come l'insieme generato dallo stato q_0 + i valori consumati dalle transizioni δ fino ad arrivare a stati finali in F .

Due NFA sono equivalenti quando i loro linguaggi accettati e rifiutati combaciano.

Domanda 41

Definizione di DFA (automa finito deterministico). Discutere cosa si intende per determinismo. Mostrare che un DFA è un caso speciale di NFA, ovvero la classe dei DFA è un sottoinsieme della classe degli NFA.

Un DFA è una quintupla $\{\Sigma, Q, \delta, q_0, F\}$ dove:

- Σ : input
- Q : stati totali
- δ : funzione di transizione
- $q_0 \subseteq Q$: stati iniziali
- $F \subseteq Q$: stati finali

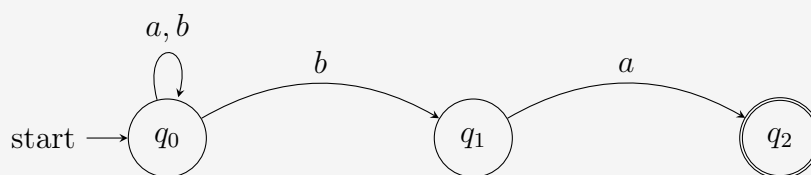
Ma la cardinalità di tutte le transizioni è 1, $|\delta(q, a)| = 1$.

Per determinismo definiamo che ad ogni stato non possiamo scegliere fra alternative ma abbiamo una strada singola.

Domanda 42

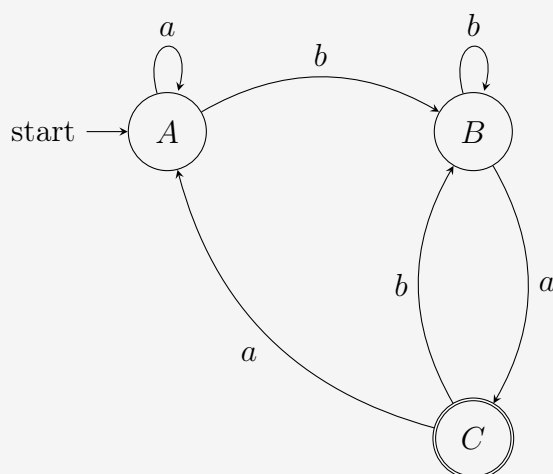
Dato un NFA, come si ricava un equivalente DFA? Descrivere la costruzione per sottoinsiemi. Definizione di epsilon-closure e algoritmo associato. Qual e la complessita della costruzione per sottoinsiemi nel caso pessimo?

Per dimostrare che $DFA \subset NFA$, dobbiamo utilizzare la ϵ -closure.



Ora calcoliamo i nuovi stati grazie alla funzione di transizione Δ del DFA:

- $A = \epsilon\text{-closure}(0): \{0\}$
- $\Delta(A, a) = \{\epsilon\text{-closure}(\underbrace{\text{mossa}(A, a)}_{\text{esclusa partenza}})\} = \epsilon\text{-closure}(\{0\}) = \{0\}$
- $\Delta(A, b) = \epsilon\text{-closure}(\{0, 1\}) = \{0, 1\} = B$
- $\Delta(B, a) = \epsilon\text{-closure}(\{0, 2\}) = \{0, 2\} = C$
- $\Delta(B, b) = \epsilon\text{-closure}(\{0, 1\}) = \{0, 1\} = B$
- $\Delta(C, a) = \epsilon\text{-closure}(\{0\}) = \{0\} = A$
- $\Delta(C, b) = \epsilon\text{-closure}(\{0, 1\}) = \{0, 1\} = B$



DFA ottenuto per costruzione per sottoinsiemi.

La ϵ -closure di q e' l'insieme degli stati raggiungibile da q solo con mosse ϵ .
Se con un NFA ha cardinalita n allora il suo DFA nel caso peggiore ha 2^n di cardinalita'.

Domanda 43

Enunciare il teorema che dice che la classe dei linguaggi riconosciuti da NFA coincide con la classe dei linguaggi riconosciuti da DFA.

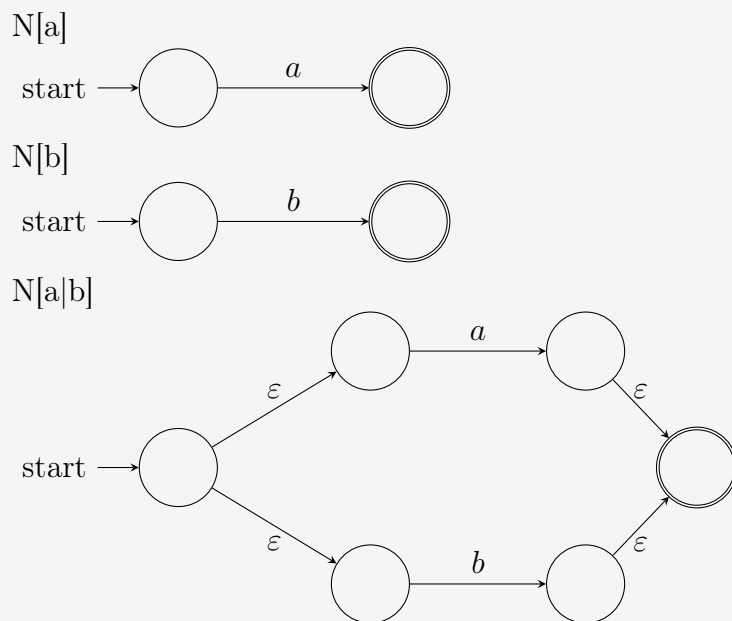
Teorema: Sia $N = (\Sigma, Q, \delta, q_0, F)$ un NFA e sia M_N l'automa ottenuto per sottoinsiemi. Allora M_N e' un DFA e si ha che:

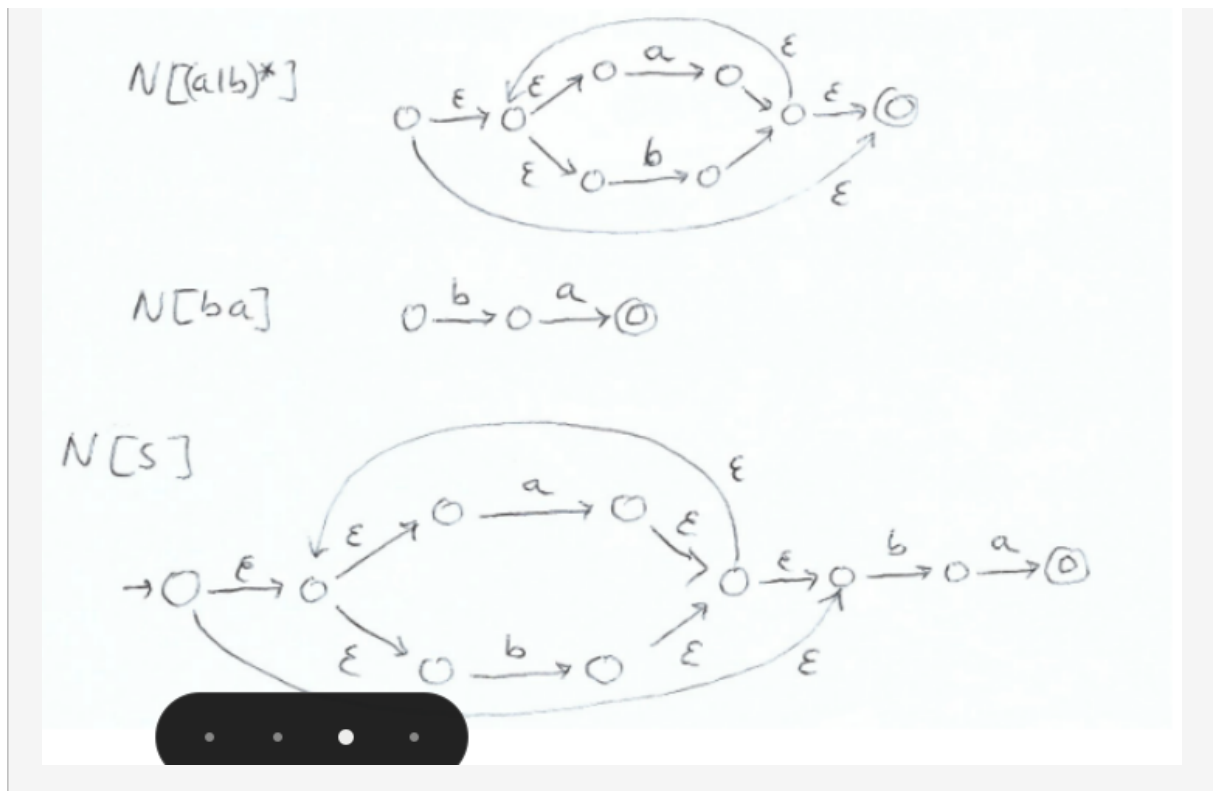
$$L[N] = L[M_N] \quad (1)$$

Domanda 44

Come si costruisce un NFA a partire da una espressione regolare, in modo tale che il linguaggio riconosciuto dall'NFA sia lo stesso del linguaggio associato all'espressione regolare?

Dato $S = (a|b)^*ba$
Partiamo dalla radice (bottom-up):





Domanda 45

Definizione di grammatica regolare.

La dice grammatica G e' regolare sse e' generata da un linguaggio regolare.
ESEMPIO:

$$G := \begin{cases} A \rightarrow aB \mid \epsilon \\ B \rightarrow bA \mid \epsilon \end{cases}$$

$$G := A \rightarrow abA \mid a \mid \epsilon$$

$$G := (ab)^*(a \mid \epsilon)$$

Domanda 46

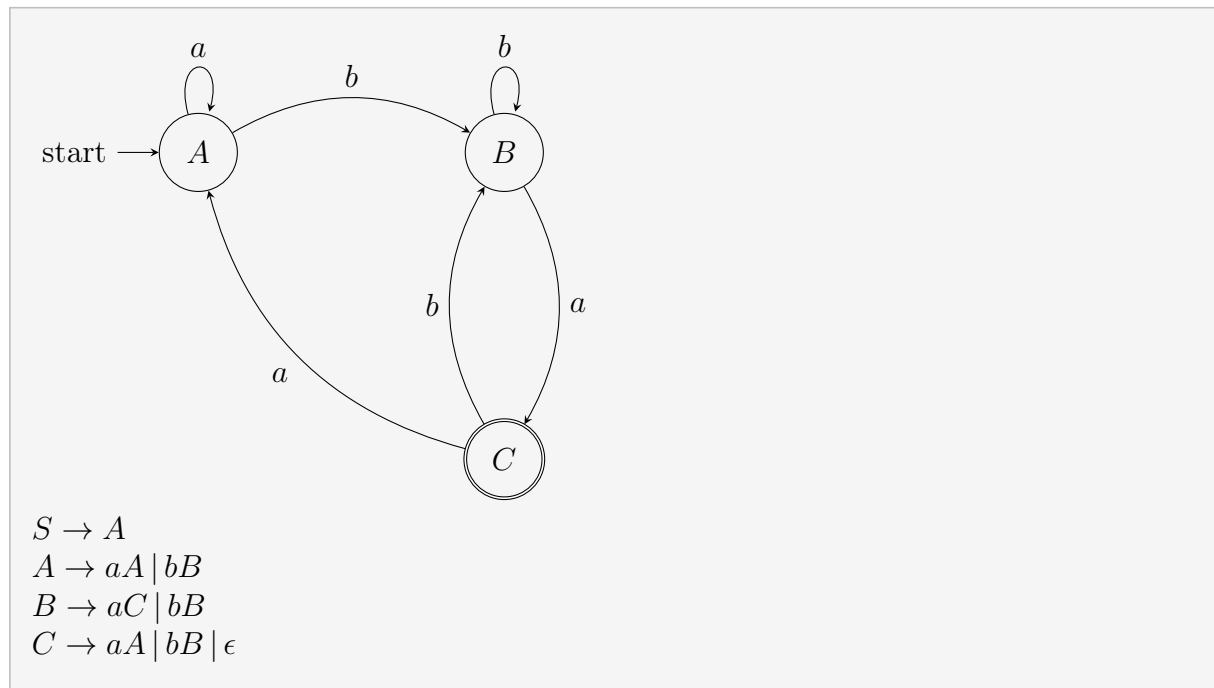
Come si associa ad una grammatica regolare un equivalente NFA?

$$NFA \Rightarrow DFA \Rightarrow \text{Grammatica regolare}$$

Domanda 47

Dato un DFA, come si costruisce una grammatica regolare equivalente?

I nodi saranno i terminali, mentre gli archi i non terminali.



Domanda 48

Data una grammatica regolare, come si costruisce un'espressione regolare equivalente?

Semplifichiamo la grammatica rimuovendo i simboli inutili

$A \rightarrow aA \mid \epsilon \mid bc$

$C \rightarrow bC \mid \epsilon \mid aD$

$D \rightarrow aD \mid bD$ (questa non finisce mai)

diventa

$A \rightarrow aA \mid \epsilon \mid bC$

$C \rightarrow bC \mid \epsilon$

Ora ricaviamo l'espressione regolare associata:

$C = b^*$

$A = a^*b^*$

Domanda 49

Descrivere, con un diagramma riassuntivo, tutte le relazioni fra i formalismi introdotti: NFA, DFA, grammatiche regolari, espressioni regolari.

$$\begin{array}{ccc}
\text{Grammatiche regolari} & \Rightarrow & \text{NFA} \\
\Uparrow & & \Downarrow \\
\text{Espressioni regolari} & \Leftarrow & \text{DFA}
\end{array}$$

Domanda 50

Definizione di stati equivalenti in un DFA. Quando due stati di un DFA sono distinguibili?

Due stati sono equivalenti quando per ogni stringa $w \in \{\Sigma + \epsilon\}$:

$$\begin{aligned}
\delta^*(q_0, w) \in F & \iff \delta^*(q_1, w) \in F \\
L[\text{DFA}, q_0] &= L[\text{DFA}, q_1]
\end{aligned}$$

Nota che δ^ e' la funzione di transizione completa su tutta la stringa.*

Due stati sono distinguibili quando:

$$\delta^*(q_0, w) \in F \text{ e } \delta^*(q_1, w) \notin F$$

Domanda 51

Come funziona l'algoritmo iterativo con tabella a scala per produrre le classi di equivalenza di stati di un DFA?

Dato un DFA con $q_0, \dots, q_i$ stati e q_i stato terminale, costruiamo la tabella a scala:

- 1. Le x sara' numerato da 0 a i - 1*
- 2. Le y sara' numerato da i a 1*
- 3. Flag su l'incrocio fra tutti gli elementi con gli stati terminali*
- 4. Controlliamo per ogni cella , ip. $q_k q_m$, ogni movimento per ogni lettera, se otteniamo anche solo una nuova coppia che e' flaggata allora flaggiamo anche qua*
- 5. Se abbiamo delle coppie di stati non flaggate allora sono equivalenti*
- 6. Costruiamo un nuovo grafo unendo le coppie equivalenti*
- 7. Ricontrolliamo*

Domanda 52

Una volta determinate le classi di equivalenza degli stati di un DFA, come si costruisce l'automa minimo associato?

1. Date le coppie di stati di equivalenza
2. Costruiamo un nuovo grafo unendo le coppie identica fra le loro relazioni
3. Ricostruiamo un nuovo DFA
4. Controlliamo se questo sia minimo

Ora abbiamo un DFA minimo!

Domanda 53

Cos'è Lex? Qual è il suo input e il suo output?

Lex è un generatore di Lexer:

- Input: file `.l` nel quale specifichiamo le definizioni regolari
- Output: file `.c` che realizza l'automa riconoscitore

Domanda 54

Qual è la struttura di un file `.l`? Come funziona l'analizzatore lessicale prodotto da Lex?

E' diviso in 3 parti:

1. Dichiarazioni (definizioni regolari)
2. Regole (pattern $\Rightarrow$ C code)
3. Funzioni ausiliarie

L'automa in C implementa il DFA riconoscitore delle espressioni regolari e restituisce `<TOKEN, lessema>`.

Domanda 55

Come si interfaccia Lex con Yacc?

Yacc chiama Lex attraverso una funzione `yylex()`, lex ritorna la tuple `<TOKEN, lessema>` e YACC è un generatore di parser (analizzatore sintattico).

Domanda 56

Intestazione e dimostrazione del pumping lemma.

Se L e' un linguaggio regolare allora $\exists N \geq 1$ t.c. per ogni stringa accettata in L ($uvw \in L$) e' $|uvw| \geq N$:

- $|uv| \leq N$
- $v \geq 1$
- $\forall k \geq 0 \quad uv^k w \in L$

DIMOSTRAZIONE: Sia $DFA = (\Sigma, Q, \delta, q_0, F)$ che genera L :

$$|DFA| = N \quad (2)$$

Consideriamo una stringa $|uvw| \geq N$, durante la lettura delle prime N lettere siamo passati da $N+1$ stati (anche iniziale), ed essendo un DFA per definizione abbiamo solo N stati. Quindi in uno stato ci siamo passati piu' volte.

Chiamiamo q_i lo stato passato piu' volte, definiamo che

- x e' la parte di string fino a q_i
- v e' la parte di loop
- w e' la parte post q_i

Ovvero che allo stato q_i se leggiamo certi caratteri (quelli dentro v) otteniamo

$$\delta^*(q_i, v) = q_i$$

la lunghezza di v non importa tanto e' un loop che consuma tutti i caratteri compatibili con il loop, ovvero

$$uv^k w \in L \quad \forall k \geq 0$$

Domanda 57

Come si puo utilizzare il pumping lemma (a rovescio) per dimostrare che un linguaggio non e regolare?

Dimostriamo che $L = \{a^n b^n | n \geq 1\}$ non sia regolare.

Assumiamo per assurdo che L sia regolare, allora vale il pumping lemma che ci dice:

- Fissata una lunghezza $N \geq 1$
- Data la stringa $uvw \in L$ e $|uvw| \geq N$:
 1. $|uv| \leq N$
 2. $|v| \geq 1$
 3. $\forall k \geq 0 \quad uv^k w \in L$

La nostra stringa e' $a^N b^N$, quindi $uv = a^N$ e $w = b^N$.

Scomponiamo $uv = a^N$ in $u = a^m$ e $v = a^{N-m}$ ($m < N$), pero' per n.3 sappiamo

che

$$\begin{aligned} uv^k w &\in L \\ \Rightarrow a^m a^{k(N-m)} b^N &\in L \quad \forall m < N \quad \forall k \geq 0 \\ \Rightarrow a^m a^{kN-km} b^N &\end{aligned}$$

Ipotizziamo $N = 2, k = 2$:

$$\Rightarrow a^m a^{4-2m} b^2$$

Ipotizziamo $m = 1$

$$\begin{aligned} \Rightarrow a^1 a^2 b^2 \\ \Rightarrow aaabb \notin L \end{aligned}$$

$\rightarrow$ assurdo! Quindi L non e' regolare.

Domanda 58

Quali sono le proprieta di chiusura dei linguaggi regolari? Quali proprieta si possono decidere (ovvero verificare algoritmicamente)?

La classe regolare e' chiusa per tutto, ovvero unione, intersezione, concatenazione, Kleene, complemento.

1.4 Capitolo 4 – Analisi sintattica, linguaggi liberi

Domanda 59

Cosa si intende per analisi sintattica? Cos'e un parser?

L'analisi sintattica e' la fase del compilatore dove data una lista di token si controlla che rispetti la grammatica del linguaggio (ordine dei token). Il parser e' il componente che esegue l'analisi sintattica e restituisce AST.

Domanda 60

Definizione di automa a pila nondeterministico (PDA). Definizione di configurazione (o descrizione istantanea), mossa in un passo e mossa in piu passi.

Un PDA (automa a pila non-deterministico) e' una 7-pla $(\Sigma, Q, \Gamma, \delta, q_0, \perp, F)$ dove:

- Σ : input
- Q : stati totali

- Γ : insieme simboli pila
- δ : funzione di transizione
- q_0 : stato iniziale
- $\perp \in \Gamma$: simbolo iniziale sulla pila
- F : stato finale

Domanda 61

Definizione di linguaggio accettato da un PDA per stato finale o per pila vuota. Sono equivalenti queste due modalita di riconoscimento?

Due modalita' di riconoscimento:

- *Stato finale:*

$$\{w \in \Sigma^* | (q_0, w, \perp) \vdash_N^* (q_f, \epsilon, \gamma) \quad q_f \in F\}$$

- *Pila vuota:*

$$\{w \in \Sigma^* | (q_0, w, \perp) \vdash_N^* (q_f, \epsilon, \epsilon) \quad q_f \in F\}$$

- q_0 stato iniziale
- w stringa ancora non letta
- $\perp$ pila vuota
- q_f stato finale
- ϵ lettera terminale
- γ contenuto pila (completa)

Le due modalita' sono equivalenti per un teorema.

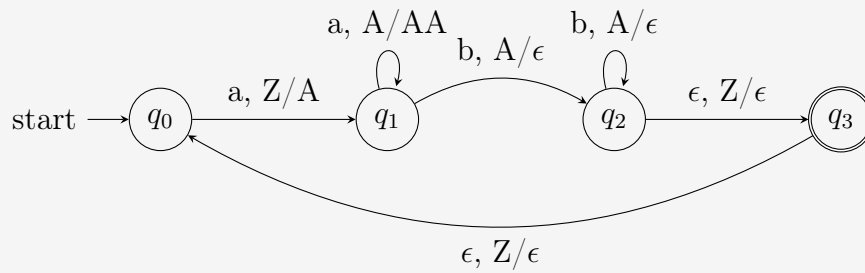
Domanda 62

Mostrare come data una grammatica libera G sia possibile costruire un PDA P equivalente. E possibile costruire, dato un PDA P , una grammatica equivalente G ? Concludere che la classe dei linguaggi liberi coincide con la classe dei linguaggi riconosciuti da PDA.

Data la grammatica libera G :

$$G = \left\{ A \rightarrow aAb \mid \epsilon \right.$$

PDA stato finale:



Si, dato il PDA qua sopra sappiamo che:

- Inizia con $(a \mid \epsilon)$
- Contiene a^*
- Contiene b^*

Quindi $L[PDA] = a^*b^*$, che si puo' scrivere come: $S \rightarrow aSb \mid \epsilon$.

Quindi abbiamo mostrato come dalla grammatica G abbiamo ottenuto un PDA che a sua volta ha generato una grammatica, tutte queste coincidono nello stesso linguaggio.

Domanda 63

Quali sono le proprieta di chiusura dei linguaggi liberi? L'intersezione di un linguaggio libero con un regolare e un linguaggio libero?

I linguaggi liberi sono chiusi per:

- Unione
- Concatenazione
- Kleene

Mentre se l'intersezione tra un libero e un regolare e' sempre libero (chiuso).

Domanda 64

Intestazione e dimostrazione del Pumping Theorem per i linguaggi liberi.

Fissiamo $N \geq 1$, allora $uvwx y \in L$ e $|uvwx y| \geq N$:

- $|vwx| \leq N$
- $|vx| \geq 1$
- $\forall k \geq 0 \quad uv^kwx^ky \in L$

Domanda 65

Come si può utilizzare tale teorema (a rovescio) per dimostrare che un linguaggio non è libero?

Ipotizziamo per assurdo che L sia libero, allora $\forall N \geq 1$ e $\forall uvwxy \in L$ t.c. $|uvwxy| \geq N$. Allora vale per pumping theorem libero:

- $|vwx| \leq N$
- $|vx| \geq 1$
- $\forall k \geq 0 \quad uv^kwx^ky \in L$

*Fissiamo un valore k t.c. $uv^kwx^ky \notin L$
 $\Rightarrow L$ non è libero!*

Domanda 66

Classificazione di Chomsky delle grammatiche e dei linguaggi. Definizione di grammatica dipendente dal contesto e di grammatica monotona. Quale tipo di automi corrisponde ad ogni classe?

1. Grammatiche regolari (DFA-NFA)
2. Grammatiche libere da contesto (PDA)
3. Grammatiche dipendenti da contesto (LBA):

$aA \rightarrow B$ Nota come dipende da 2 argomenti a sinistra!

4.
 - Grammatiche Monotone: non acconsente di accorciare ($aA \rightarrow B$)

$A \rightarrow B$ con $|A| \leq |B|$

- Grammatiche generali (Macchine di Turing)

Domanda 67

Definizione di DPDA (automa a pila deterministico) e di linguaggio libero deterministico.

Un linguaggio è libero deterministico se è accettato per stato finale da un DPDA:

$$\forall q \in Q \quad \forall z \in \Gamma \quad \forall a \in \Sigma^* \quad |\delta(q, a, z)| \leq 1$$

Ovvero ad ogni stato se dobbiamo mettere a dentro la pila possiamo prendere 1 sola

strada possibile.

Domanda 68

La classe dei linguaggi liberi deterministici è strettamente inclusa in quella dei linguaggi liberi? Contiene strettamente la classe dei linguaggi regolari?

$$\begin{aligned} REG &\subset DCFG \subset CFG \\ NFA/DFA &\subset DPDA \subset PDA \end{aligned}$$

Domanda 69

Che cosa dice la prefix property e perché è interessante per i DPDA?

Un linguaggio libero deterministico gode della prefix property quando nessuna parola è prefisso di un'altra.

Tipo $L = \{a, ab\}$ il DPDA dopo aver letto a non saprebbe se chiudere o continuare. Quindi non sarebbe deterministico.

Domanda 70

Usando un endmarker $\$$, si può riconoscere un linguaggio libero deterministico che non gode della prefix property anche per pila vuota? Come?

Esattamente $\$$ serve proprio per far riconoscere per pila vuota un DPDA che non gode di prefix property.

Inseriamo il simbolo $\$$ al fine del linguaggio:

$$L \rightarrow L\$$$

Così quando leggiamo $\$$ siamo sicuri che sia finito.

Domanda 71

Un linguaggio libero deterministico è ambiguo?

Non sempre dipende, un DPDA è deterministico ma possono esistere grammatiche ambigue che generano il linguaggio.

Domanda 72

Proprietà di chiusura dei linguaggi liberi deterministici: chiusi per complementazione, ma non per intersezione né per unione.

- Complemento
- Intersezione con i regolari

Domanda 73

Da cosa si parte per costruire un analizzatore sintattico (ovvero parser)? Da una espressione regolare? Da una grammatica libera? Da un PDA?

Si utilizza una CFG. Il PDA lo utilizziamo solo come modello per il riconoscimento. L'espressione regolare non va bene perché è limitante esprimere un linguaggio regolare (NFA/DFA).

Domanda 74

Cosa prende in input e cosa produce in output un parser?

Prende in input una tupla $\langle \text{TOKEN}, \text{lessema} \rangle$ e restituisce in output un AST.

Domanda 75

Che differenza c'è tra un parser nondeterministico ed uno deterministico?

Il parser deterministico è quello "classico" che prende scelte assolute, mentre quello nondeterministico utilizza il backtracking quando una scelta è errata.

Domanda 76

Quali sono le due tecniche essenziali per costruire parsers?

Le due tecniche sono:

- Top-down: data la stringa in input si partì dal simbolo iniziale S
- Bottom-up: data la stringa in input si cerca di ridurre fino a S

Domanda 77

Le tecniche top-down e bottom-up in che cosa differiscono?

- *Top-down: partiamo con S sulla pila e la espandiamo a seconda dell'input*
- *Bottom-up: partendo dalla pila vuota pushiamo (shift) gli input sulla pila, quando poi abbiamo una combinazione li traduciamo in Terminali (reduce) e continuiamo fino ad avere stato iniziale S*

Domanda 78

Quali tipi di grammatiche non sono adatte al top-down parsing? Quali tipi di produzioni sono poco adatte al bottom-up parsing?

Il Top-down non supporta il left recursion

$$A \rightarrow Aa \mid b$$

Il Bottom-up ha problemi per i conflitti shift/reduce:

$$E \rightarrow E + E \mid id$$

Top-down e Bottom-up DETERMINISTICI non supportano le ϵ -produzioni:

$$A \rightarrow \epsilon$$

Domanda 79

Cosa sono le produzioni epsilon e cosa sono i simboli non-terminali annullabili?

Le ϵ -produzioni sono la riduzione da

$$A \rightarrow \epsilon$$

$$L(G) = L(G) \setminus \{\epsilon\}$$

non sono compatibili con i DPDA.

I simboli non-terminali annullabili sono quelli che si riducono a ϵ -produzioni (anche indirettamente).

Domanda 80

Come si può trasformare una grammatica G che contiene produzioni epsilon in una grammatica G' che non ne contiene, preservando il linguaggio a meno di epsilon?

Se $\epsilon \in L(G)$ e vogliamo $L(G') = L(G) \setminus \{\epsilon\}$ allora:

- Eliminiamo i non-terminali annullabili
- Mettiamo un nuovo simbolo iniziale $S' \rightarrow S \mid \epsilon$

Domanda 81

Cosa sono le produzioni unitarie e cosa sono le coppie unitarie?

Dati i non-terminali A, B li definiamo unitari se

$$A \rightarrow B$$

Li definiamo coppie unitarie se

$$A \rightarrow^* B \text{ attraverso solo coppie unitarie}$$

Domanda 82

Come si può trasformare una grammatica G che contiene produzioni unitarie in una equivalente G' che non ne contiene?

Possiamo sostituire tutte le produzioni unitarie $A \rightarrow B$ direttamente con le produzioni di B .

Domanda 83

Data una grammatica G , quali sono i suoi simboli utili? Quali sono i suoi simboli generatori? Quali i suoi simboli raggiungibili?

Un non-terminale è generatore quando riesce a derivare almeno una stringa di soli terminali

$$A \rightarrow^* w \quad w \in \Sigma^*$$

Un simbolo (terminale e non) è raggiungibile quando si può produrre partendo da S .
Un simbolo è utile quando è generatore e raggiungibile.

Domanda 84

Come si calcolano i generatori? Come si calcolano i raggiungibili?

I generatori si calcolano:

- Segniamo tutti i non-terminali che si riducono con 1 step a terminale (stringhe)
- Segniamo ora tutti i non-terminali che si riducono a i non-terminale di prima
- Così via
- I restanti alla fine sono i non generatori.

I simboli non raggiungibili si calcolano:

- Iteriamo da S ogni possibile strada presa dalle riduzioni
- Quando arriviamo a un terminale (stringhe) lo flaggiamo
- Backtracking alla divarazione precedente
- Ripetiamo finché non abbiamo passato tutte le divariazioni
- I non flaggati sono non raggiungibili

Domanda 85

In che modo si eliminano i simboli inutili di una grammatica? È importante l'ordine delle operazioni da svolgere?

Calcoliamo i simboli non-terminali generatori, quelli non-generatori li cancelliamo ovunque.

Calcoliamo i simboli raggiungibili, i restanti li cancelliamo. Ora abbiamo solo simboli utili.

Domanda 86

Quando si dice che una grammatica è ricorsiva sinistra? Come si elimina la ricorsione sinistra immediata? E quella non immediata? Perché serve eliminare la ricorsione sinistra?

Una grammatica è ricorsiva sinistra (left recursion) quando abbiamo

$$A \rightarrow Aa$$

Ovvero il primo carattere della riduzione è il non-terminale stesso che si è appena ridotto.

Per rimuoverla se abbiamo:

$$A \rightarrow Aa|B|$$

Definiamo un nuovo non-terminale per produrre ricorsivamente a destra "a" chiamato

A'

$$A' \rightarrow aA' \mid \epsilon$$

e modifichiamo A in modo che prima chiamiamo B e poi generiamo gli "a":

$$\begin{aligned} A &\rightarrow BA' \\ A' &\rightarrow aA' \mid \epsilon \end{aligned}$$

Domanda 87

Cosa vuol dire fattorizzare a sinistra una grammatica? Perché serve fattorizzare?

In un DPDA se abbiamo

$$A \rightarrow aB \mid a$$

non sappiamo cosa scegliere. Quindi creiamo un nuovo terminale A' che ci genera la parte non-comune:

$$\begin{aligned} A &\rightarrow aA' \\ A' &\rightarrow B \end{aligned}$$

Questo ci permette di avere lo stesso linguaggio ma con una grammatica che vada su un parser deterministico.

Domanda 88

Cos'è un parser a discesa ricorsiva?

Un parser a discesa ricorsiva è un parser top-down che ricostruisce partendo da S attraverso dichiarazioni ricorsive.

Per il linguaggio

$$\begin{aligned} E &\rightarrow T + E \mid T \\ T &\rightarrow id \end{aligned}$$

lo definiamo con:

$$\begin{aligned} E() : & \quad T() \\ & \quad \text{if next} == '+' \text{ then} \\ & \quad \quad \text{match}(' + ') \\ & \quad \quad E() \\ T() : & \quad \text{match}(id) \end{aligned}$$

Domanda 89

Definizione di First(a) e di Follow(A).

First e' l'insieme dei terminali che sono in prima posizione dal terminale A :

$$A \rightarrow aA \mid \epsilon$$
$$\text{First}(A) = \{a, \epsilon\}$$

Follow del non-terminale X e' l'insieme dei terminali che spuntano dopo per definizione o riduzione ad X :

$$S \rightarrow Ab \mid c$$
$$A \rightarrow aA \mid \epsilon$$
$$\text{Follow}(S) = \{\$ \}$$
$$\text{Follow}(A) = \{b\}$$

Domanda 90

Algoritmi per calcolare First(a) e Follow(A).

First:

- Per ogni terminale x facciamo che $\text{First}(x) = \{x\}$
- Per ogni non-terminale A :
 - Per ogni sua produzione x :
 - * Per ogni carattere di x chiamato c :
 - $\text{First}(A)^+ = \text{First}(c)$

Follow:

- $\text{Follow}(S) = \{\$ \}$
- Per ogni non-terminale A :
 - Per ogni sua produzione x :
 - * Per ogni carattere di x chiamato c :
 - if $A == S$ and c is ultimo in x : $\text{Follow}(A)^+ = \{\epsilon\}$
 - else if c is not ultimo in x : $\text{Follow}(c)^+ = \text{First}(c+1) \setminus \{\epsilon\}$

Domanda 91

Come e fatta e come si riempie una tabella di parsing LL(1)?

Righe non-terminali, Colonne terminali.

Dalle produzioni per ogni per ogni $c \in \text{First}(A)$ compiliamo la cella (A, c) con la produzione $A \rightarrow x \dots$

Mentre i $c \in \text{Follow}(A)$ ci servono per ridurre $A \rightarrow \epsilon$ quando leggiamo c .

Domanda 92

Quando una grammatica G si dice di classe $LL(1)$? Quali sono le condizioni necessarie e sufficienti per G affinché sia di classe $LL(1)$?

Una grammatica G è $LL(1)$ se la sua tabella di parsing $LL(1)$ contiene al max. 1 produzione per ogni casella.

E' $LL(1)$ quando per ogni

$$A \rightarrow B \mid C$$

abbiamo

- 1. $\text{First}(B) \cap \text{First}(C) = \emptyset$*
- 2. Se $\epsilon \in \text{Follow}(B)$ allora $\text{Follow}(A) \cap \text{First}(C) = \emptyset$*
- 3. Se $\epsilon \in \text{Follow}(C)$ allora $\text{Follow}(A) \cap \text{First}(B) = \emptyset$*

Nota che 2/3 servono perché se avessimo caratteri in comuni non sapremmo se ridurre $A \rightarrow C \rightarrow \epsilon + B \rightarrow c$ oppure $A \rightarrow B \rightarrow c$

Domanda 93

Perché un parser di questo tipo è chiamato LL ?

Left-to-right Leftmost

Perché è un parser top-down che partendo da S consuma da sinistra a destra gli input per arrivare allo stato finale.

Domanda 94

Come funziona il parser $LL(1)$ con una pila?

A seconda di quello che è il carattere in lettura pushiamo nello stack la produzione della sua cella in base al non terminale dello stack per poi consumarlo da entrambi i lati. Continiamo finché non abbiamo più caratteri in input e la pila con il terminatore.

Domanda 95

E vero che ogni linguaggio regolare e pure LL(1)?

Si perché ogni grammatica regolare può essere trasformata in una LL(1) compatibile, dato che le sue restrizioni forti permettono di non avere mai conflitti in LL(1).

Domanda 96

Come sono definiti $\text{First}_k(a)$ e $\text{Follow}_k(A)$ per $k \geq 2$?

Le definizioni di FIRST_k e FOLLOW_k estendono FIRST e FOLLOW considerando stringhe terminali di lunghezza al più k .

FIRST_k : Per una stringa α , $\text{FIRST}_k(\alpha)$ è l'insieme delle stringhe terminali w tali che:

$$\alpha \Rightarrow^* w\beta \quad \text{con } |w| \leq k$$

cioè tutte le possibili stringhe di terminali (lunghezza $\leq k$) che possono apparire come prefisso di una derivazione di α .

FOLLOW_k : Per un non-terminale A , $\text{FOLLOW}_k(A)$ è l'insieme delle stringhe terminali w tali che:

$$S \Rightarrow^* \alpha A w \beta \quad \text{con } |w| \leq k$$

cioè tutte le stringhe di terminali (lunghezza $\leq k$) che possono comparire immediatamente a destra di A in una derivazione.

Domanda 97

Come si definisce una grammatica LL(k)? Ed un linguaggio LL(k)? Come si relazionano tra di loro?

LL(k) è una quadrupla (NT, T, δ, S) con:

- NT insieme non terminali
- T insieme terminali
- δ funzione transazione
- S simbolo iniziale

Un linguaggio $L \subseteq \Sigma^$ è LL(k) se esiste almeno una grammatica LL(k) che lo genera.*

Domanda 98

Che relazione esiste tra grammatiche LL(k) e grammatiche ambigue? E con le grammatiche ricorsive sinistre?

Una grammatica $LL(k)$ non e':

- *ambigua*
- *ricorsiva sinistra*

Ogni grammatica $LL(k)$ non e' ambigua e ogni grammatica ambigua e' non $LL(k)$.

Domanda 99

Esistono linguaggi liberi che non sono $LL(k)$ per nessun k ? Ed esistono linguaggi liberi deterministici che non sono $LL(k)$ per nessun k ?

Si perche' abbiamo che

$$LL(k) \subset DCFG \subset CFG$$

Quindi essendo una relazione stretta alcuni linguaggi non esistono nel sottoinsieme minore.

Domanda 100

Cos'e un parser bottom-up (o shift-reduce)? Qual e il suo input e il suo output? Perche sono chiamati parser LR?

Un parser bottom-up parte dalla pila vuota e attraverso gli shift-reduce arriviamo allo stato iniziale S .

Il suo input sono una lista di token, e il suo output e' un ast.

Si chiama LR perche' e left-to-right rightmost, ovvero legge da sinistra a destra ma partiamo dalla pila vuota.

Domanda 101

Che tipo di conflitti si possono presentare in un parser del genere? Quando si presenta un conflitto (shift/reduce o reduce/reduce), quale azione bisogna scegliere?

Possiamo avere conflitto shift-reduce o reduce-reduce ovvero momenti che il parser deve scegliere non deterministicamente una strada.

Se la grammatica e' $LR(k)$ allora il problema shift-reduce o reduce-reduce non si pone.

Domanda 102

Cos'è un prefisso viabile? Come lo si definisce in termini di una grammatica?

Le stringhe che otteniamo durante il parsing bottom-up corretto si chiamano prefissi viabili.

Data la grammatica $G = (NT, T, \delta, S)$, definiamo prefisso viabile:

$$\{ a \in (NT \cup T)^* \mid \exists \beta, A, w, \gamma : S \Rightarrow^* \beta A w \Rightarrow \beta \gamma w \wedge a \preceq \beta \gamma \}.$$

Domanda 103

Cos'è un item LR(0)? Come si generano tutti gli item di una grammatica (aumentata con un simbolo iniziale nuovo S')?

Gli item in LR(0) indicano quali prefissi viabile abbiamo riconosciuto e quale simbolo ci stiamo aspettando.

Dato $S \rightarrow A_1 \dots A_n$

Generiamo un item per ogni posizione:

- $S' \rightarrow .S$
- $S' \rightarrow S.$
- $S \rightarrow .A_1 \dots A_n$
- $S \rightarrow A_1 \dots A_n.$
- $\vdots$
- $S \rightarrow A_1 \dots A_n.$
- $A \rightarrow .a$
- $A \rightarrow a.$

Se abbiamo $S \rightarrow \epsilon$ mettiamo $S \rightarrow ..$

Nota come se abbiamo dopo un punto un terminale espandiamo anche lui.

Domanda 104

Come è fatto il NFA dei prefissi viabili? Come si ricava il DFA dei prefissi viabili, detto anche automa canonico LR(0)?

Data la grammatica estesa G :

- $q_0 = [S' \rightarrow .S]$
- $q_i =$

- Se abbiamo un NT mettiamo una transizione a costo ϵ tipo $[A \rightarrow B.a]$
- Se abbiamo un T mettiamo una transizione a costo a tipo $[A \rightarrow a.B]$

Quindi per ogni transizione in G abbiamo uno stato nel NFA .

Il DFA (automa canonico $LR(0)$) lo otteniamo:

1. Costruzione per sottoinsiemi
2. $Clos(x)$ e $Goto(I, x)$

Domanda 105

Come e fatta una tabella di parsing $LR(0)$? Come la si riempie a partire dall'automa canonico $LR(0)$? Quando una grammatica e di classe $LR(0)$?

Dato DFA automa canonico $LR(0)$ facciamo una tabella

- righe: 1 fino a $|DFA|$
- colonne: $T \cup NT \cup \{\$ \}$

Se dallo stato q_i abbiamo un T che vada a k allora nella casella (i, T) mettiamo $goto(k)$.
Se dallo stato q_i abbiamo un NT che vada a k allora nella casella (i, NT) mettiamo $shift(k)$.

Una grammatica e' $LR(0)$ quando la sua tabella di parsing $LR(0)$ non ha conflitti.

Domanda 106

Come e fatto il parser $LR(0)$ che utilizza la tabella di parsing $LR(0)$? Quanti stack servono?

- Automa che genera tabella $LR(0)$
- 1 Stack che indica il percorso degli stati (tipo $[0,2,3]$)
- Tabella compilata $LR(0)$
- Input

Domanda 107

Esistono grammatiche non $LR(0)$? Fare un esempio semplice.

Si non avendo lookhaed non riesci a risolvere:

$$S \rightarrow aS \mid a$$

oppure con ricorsione sinistra:

$$S \rightarrow Sa \mid a$$

Non sappiamo se leggiamo "a" se derivi da Sa o a.

Questo perche' $First(S) \cap Follow(S) \neq \emptyset$.

Domanda 108

Come e fatta e come si riempie una tabella di parsing SLR(1)? Cosa vuol dire l'acronimo SLR? Perche si mette 1 come parametro?

Attraverso 1 parametro di lookhaed riusciamo ad ampliare il linguaggio di LR(0) riducendo i conflitti shift-reduce. Infatti

$$LR(0) \subset SRL(1)$$

Partendo da automa DFA LR(0) riusciamo a modificarlo attraverso 1 simbolo di lookhaed rispettivamente $Follow_1$.

*SLR significa: "simple left-to-right rightmost", diciamo **simple** perche rispetto LR(1) la tecnica di seguire i Follow e' banale e meno precisa.*

Per definizione ci serve almeno 1 parametro, infatti SLR(0) non esiste.

Domanda 109

Quando una grammatica e di classe SLR(1)? Esistono grammatiche non di classe SLR(1)?

Definiamo $G = (NT, T, \delta, S)$ grammatica SLR(1) se la sua tabella di parsing LR(0) + 1 simbolo di lookhaed basata sui i Follow non contiene conflitti.

Si, esistono grammatiche non SLR(1) anche qua possiamo avere conflitti shift-reduce.

Domanda 110

Cos'e un item LR(1)? Come si costruisce il NFA LR(1)? E come l'automa canonico LR(1)?

Un item LR(0) e' tipo:

$$A \rightarrow a.$$

mentre un item $LR(1)$ e'

$$A \rightarrow a., a$$

Ovvero possiamo produrre $A \rightarrow a$ solo se il prossimo simbolo e' un "a".

NFA $LR(1)$:

- $q_0 = [S' \rightarrow .S, \$]$
- $q_i =$
 - Da $[A \rightarrow \omega.xB, a]$ abbiamo una transizione di costo $x \in T$ per andare allo stato $[A \rightarrow \omega x.B, a]$
 - Da $[A \rightarrow \omega.xB, a]$ abbiamo una transizione di costo ϵ per andare allo stato $[A \rightarrow \omega x.B, a]$ per ogni produzione di x

Dato NFA $LR(1)$ otteniamo il DFA $LR(1)$ (automa canonico $LR(1)$) in due modi:

1. Costruzione per sottoinsiemi
2. $Clos(x)$ e $Goto(I, x)$

Domanda 111

Come e fatta e come si riempie una tabella di parsing $LR(1)$?

Dato DFA $LR(1)$ (automa canonico $LR(1)$) facciamo la tabella:

- righe: 1 fino a $|DFA|$
- colonne: $T \cup NT \cup \{\$\}$

Se dallo stato q_i abbiamo un T che vada a k allora nella casella (i, T) mettiamo $goto(k)$.
Se dallo stato q_i abbiamo un NT che vada a k allora nella casella (i, NT) mettiamo $shift(k)$.

Domanda 112

Come e fatta una tabella di parsing LALR(1)? Quando una grammatica e di classe LALR(1)?

Partendo dall'automa $LR(1)$ si fondono gli stati con *core* identico: Dati due stati

dall'automa DFA LR(1):

$$\begin{aligned} \text{Stato1} &:= \\ &\quad [A \rightarrow \alpha.\beta, a] \\ &\quad [B \rightarrow \gamma., b] \end{aligned}$$
$$\begin{aligned} \text{Stato2} &:= \\ &\quad [A \rightarrow \alpha.\beta, c] \\ &\quad [B \rightarrow \gamma., d] \end{aligned}$$

Stato 1 e 2 hanno lo stesso core $\Rightarrow$ in LALR(1) li fondiamo.

$$\begin{aligned} \text{Stato1/2} &:= \\ &\quad [A \rightarrow \alpha.\beta, a/c] \\ &\quad [B \rightarrow \gamma., b/d] \end{aligned}$$

Nella tabella gli shift/goto rimangono uguali bisogna solo aggiornare gli indici nuovi con quelli fusi.

Una grammatica $G = (NT, T, \delta, S)$ e' di classe LALR(1) quando la tabella di parsing LALR(1) non ha conflitti.

Domanda 113

Esistono grammatiche LALR(1) che non sono SLR(1)? Esistono grammatiche LR(1) che non sono LALR(1)? Mostrare una grammatica che e LR(1) ma non LALR(1).

Si esistono, infatti

$$SLR(1) \subset LALR(1) \subset LR(1)$$

Ecco un esempio di grammatica LR(1) e non LALR(1):

$$\begin{aligned} S &\rightarrow aEb \mid bEa \mid aFa \mid bFb \\ E &\rightarrow e \\ F &\rightarrow e \end{aligned}$$

Domanda 114

Come si puo generalizzare l'idea per ogni $k \geq 2$? Ovvero quando una grammatica e di classe LR(k), SLR(k) o LALR(k)?

- $First_k(A) = \{w \mid A \rightarrow^* wv \quad \text{con } |w| \leq k\}$
- $Follow_k(A) = \{w \mid S \rightarrow^* Awv \quad \text{con } |w| \leq k\}$

E generalizziamo:

- $SLR(k)$ si basa su $LR(0)$ e segue per tutti i $Follow_k$
- $LR(k)$ si basa sui lookhaed esatti ad ogni istante
- $LALR(k)$ si basa su $LR(k)$ e unisce gli stati per nucleo

Domanda 115

Come si relazionano le grammatiche della famiglia LR (LR, SLR, LALR) al variare di k? Come si relazionano le grammatiche $LR(k)$ e $LL(k)$ al variare di k? Le grammatiche $LR(k)$ e $LL(k)$ sono sempre non ambigue? Esistono grammatiche che non sono $LR(k)$ per nessun k?

$$LR(k-1) \subset SLR(k) \subset LALR(k) \subset LR(k)$$

$$LL(k) \subset LR(k)$$

Essendo parser deterministici LL e LR per definizione le grammatiche sono non-ambigue.

Sappiamo che

$$LR(k) \subset CFG$$

No, non esistono grammatiche $LR(k)$ che non sono DCFL per nessun k infatti vale:

$$\bigcup_{k \geq 1} LR(k) = DCFL$$

Domanda 116

Come si relazionano i linguaggi della famiglia $LR(k)$ rispetto a quelli $LL(k)$? Esistono linguaggi liberi deterministici che non sono $LR(k)$ per qualche k? Esistono linguaggi liberi deterministici che non sono $LL(k)$ per qualche k?

$$LL(k) \subset LR(k)$$

Si esistono linguaggi liberi deterministici che non sono $LR(k)$ per qualche (non tutti)

k :

$$LR(k) \subset DCFL \subset CFG$$

Si esistono linguaggi liberi deterministici che non sono $LL(k)$ per qualche (non tutti) k :

$$LL(k) \subset LR(k) \subset DCFL \subset CFG$$

Domanda 117

Esiste un linguaggio regolare che non è $LR(0)$? Come si relazionano i linguaggi $LR(0)$ rispetto a quelli $LL(1)$? La prefix property è una condizione necessaria e sufficiente affinché un linguaggio sia di classe $LR(0)$?

$$REG \subset DCFG \subset CFG$$

$$LR(k) \subset DCFG \subset CFG$$

$$\Rightarrow REG \neq LR(k)$$

Quindi esistono linguaggi regolari ma non $LR(0)$.

$LR(0)$ e $LL(1)$ non sono compatibili in relazione.

Se la prefix property è necessaria (se no avremmo conflitti shift-reduce) per essere $LR(0)$ ma non è sufficiente.

Domanda 118

La classe dei linguaggi $SLR(1)$ coincide con la classe dei linguaggi liberi deterministici?

No

$$SLR(1) \subset LALR(1) \subset LR(1) \subset DCFL$$

Domanda 119

Cos'è YACC? Qual è il suo input e il suo output? Come si ottiene un parser eseguibile a partire da un file .y? Come agisce il parser generato da YACC in sintonia con lo scanner generato da Lex?

*Yacc e' un generatore di parser, dato un file .y dove definiamo dichiarazioni, regole e definizioni e ci restituisce il parser .c .
Il parser YACC chiama attraverso la funzione `yylex()` il lexer che restituisce i token.*

Domanda 120

Come e la struttura di un file .y di YACC? Cosa sono le regole? Cos'e l'azione semantica?

Un file .y e' diviso in:

- 1. Dichiarazioni*
- 2. Regole grammaticali ($S \rightarrow A \mid B$)*
- 3. Definizioni ausiliarie*

Le regole sarebbero la gramamtica che il parser deve seguire.

L'azione semantica è il codice C che esegui in occasione di quella riduzione — è separata e opzionale. Potresti ridurre senza fare nulla, oppure sommare (interprete), oppure costruire un nodo AST (compilatore), oppure emettere codice assembly.

Domanda 121

E possibile gestire grammatiche ambigue con YACC, specificando le associativita e le priorit  fra gli operatori per risolvere l'ambiguit ? Come si comporta YACC in presenza di conflitti?

S , YACC permette di usare grammatiche ambigue dichiarando esplicitamente precedenza e associativit  con:

```
%left '+' '-'  
%left '*' '/'  
%right '!'
```

Piu' basso = precedenza maggiore.

In presenza di conflitti YACC non si ferma — compila lo stesso ma emette un warning e applica regole di default:

- shift/reduce conflict → di default fa shift*
- reduce/reduce conflict → di default riduce con la produzione che appare prima nel file*

1.5 Capitolo 5 – Fondamenti

Domanda 122

E possibile costruire un programma *Check* che, preso in input un qualunque programma *P*, restituisce 1 se *P* e corretto e 0 se *P* e scorretto? Ovvero esiste un qualche compilatore che puo scovare tutti i possibili errori di un programma?

Non puo' esistere per l'Halting Problem.

Immaginiamo il programma "notCheck" cosi:

- *if Check(P) == 1:*
 - *loop infinito*
- *termina*

Ma se passiamo notCheck(notCheck) se Check(notCheck) rileva errori pero' notCheck terminera'.

Mentre se non rileva errori va in loop infinito (che e' un errore).

Quindi e' impossibile.

Domanda 123

Cosa dice il problema della fermata (Halting Problem)? Come si dimostra che il problema non puo essere risolto?

Dato il problema: "possiamo creare un compilatore che staticamente rilevi tutti gli errori runtime?".

Definiamo H l'algoritmo che rilevi se un programma dia errore o no:

$$H(P, input) = \begin{cases} 1 & \text{terminato correttamente} \\ 0 & \text{errore} \end{cases}$$

Definiamo $K(P) = H(P, P)$.

$$K(P) = H(P, P) = \begin{cases} 1 & P(P) \text{ terminato correttamente} \\ 0 & P(P) \text{ errore} \end{cases}$$

Definiamo $G(P) = !K(P)$

$$G(P) = !K(P) = !H(P, P) = \begin{cases} 0 & P(P) \text{ terminato correttamente} \\ 1 & P(P) \text{ errore} \end{cases}$$

Ma cosa succede con $G(G)$?

$$G(G) = !K(G) = !H(G, G) = \begin{cases} 0 & P(G) \text{ terminato correttamente} \\ 1 & P(G) \text{ errore} \end{cases}$$

Ma questo vuol dire che:

$G(G) = 0$ sse $G(G) = 1$ terminato correttamente

$G(G) = 1$ sse $G(G) = 0$ errore

Abbiamo dimostrato $1 = 0$, quindi assurdo!

G non esiste $\Rightarrow K$ non esiste $\Rightarrow H$ non esiste!

Domanda 124

Quando un problema è decidibile?

Una proprietà è decidibile se esiste un algoritmo che, dato un programma in input, risponde sempre sì o no in tempo finito.

Una proprietà è indecidibile quando non può esistere nessun algoritmo che risponda sempre correttamente. Non è questione di potenza di calcolo o tempo — è impossibile in linea di principio.

Un esempio è l'Halting Problem.

Domanda 125

Quali sono tipici esempi di proprietà indecidibili per i linguaggi di programmazione?

- *Halting Problem*
- *Equivalenza fra due programmi (dovremmo provare tutti gli input possibili, infiniti)*
- *Correttezza (per tutti gli input, infiniti)*
- *Raggiungibilità (per tutti gli input, infiniti)*
- *Errori a runtime (per tutti gli input, infiniti)*

Domanda 126

Cos'è una Macchina di Turing (MdT)? Che cosa calcola?

Una macchina di Turing è un automa $M = (Q, \Sigma, \Sigma_B, \delta, q_0, q_f)$

- *Q insieme stati*
- *Σ alfabeto input*
- *Σ_B alfabeto input del nastro*
- *δ funzione transizione*

- q_0 stato iniziale
- q_f stato finale

Calcola un risultato di funzione (sul nastro) se l'algoritmo termina correttamente.

Domanda 127

Quando un linguaggio è detto Turing-completo? Cosa afferma la tesi di Church-Turing e perché non si può dimostrare?

Quando può implementare una macchina di Turing, ovvero serve al minimo:

- Variabili
- Salti condizionati (if-else)
- While (finiti/infiniti)
- (Ricorsione)

Church-Turing ha definito che qualsiasi metodo "manuale" possa essere emulato da una macchina di Turing, non si può dimostrare perché è un concetto filosofico e non matematico. Anche se volessimo farlo dovremmo dimostrarlo su infiniti input.

Domanda 128

I normali linguaggi di programmazione sono Turing-completi? Cosa afferma il teorema di Jacopini-Bohm?

Si i linguaggi standard sono Turing completi.

Jacopini-Bohm dimostra come qualsiasi astrazione di controllo può implementata se abbiamo soltanto:

- sequenza di comandi
- if-else
- while

Ovvero il goto e ricorsione non serve.

Domanda 129

Quale relazione esiste tra espressività di un formalismo e decidibilità di proprietà dello stesso? Fare una panoramica prendendo in esame i tre formalismi MdT, PDA, DFA, e le due proprietà $w \in L(M)$ e $L(M_1) = L(M_2)$.

<i>Proprietà</i>	<i>DFA</i>	<i>PDA</i>	<i>MdT</i>
$w \in L(M)$	<i>decidibile</i>	<i>decidibile</i>	<i>solo riconoscibile</i>
$L(M_1) = L(M_2)$	<i>decidibile</i>	<i>indecidibile</i>	<i>indecidibile</i>

Membership $w \in L(M)$:

- **DFA**: si simula l'automa su w passo per passo. L'automa ha un numero finito di stati e w è finita, quindi termina sempre. *Decidibile*.
- **PDA**: analogamente si simula il PDA su w . Anche qui la computazione termina sempre (per PDA deterministici; per quelli non deterministici si esplorano tutti i rami). *Decidibile*.
- **MdT**: la macchina potrebbe non terminare mai su certi input, andando in loop infinito. Non esiste un algoritmo che risponda sempre. Il problema è solo riconoscibile: se $w \in L(M)$ la macchina prima o poi accetta, ma se $w \notin L(M)$ potrebbe non fermarsi mai.

Equivalenza $L(M_1) = L(M_2)$:

- **DFA**: si minimizzano entrambi gli automi tramite l'algoritmo di minimizzazione, ottenendo due DFA canonici. Se sono isomorfi i linguaggi sono uguali. *Decidibile*.
- **PDA**: non esiste un algoritmo generale. Si dimostra per riduzione dall'Halting Problem o da altri problemi indecidibili noti sui linguaggi context-free. *Indecidibile*.
- **MdT**: proprietà non banale della funzione calcolata, quindi indecidibile per il **Teorema di Rice**.

Relazione generale: più un formalismo è espressivo, meno proprietà su di esso sono decidibili. La potenza espressiva ha un costo: al crescere di ciò che il formalismo può calcolare, si perde la capacità di ragionare staticamente sul suo comportamento.

2 Modulo 2 – Gabbrielli

Domanda 1

Definizione di ambiente.

Insieme delle relazioni nome $\Rightarrow$ Oggetto. Attraverso le dichiarazioni creiamo una nuova relazione nell'ambiente.

Domanda 2

Cos'è un blocco?

E' un sottoinsieme dell'ambiente, l'ambiente e' diviso in blocchi a seconda di dove abbiamo definito le dichiarazioni:

- *Ambiente globale: non e' riferito a nessun blocco*
- *Ambiente locale: definito nel blocco*
- *Ambiente non locale: ereditato da un'altro blocco*

Domanda 3

Differenza tra scope statico e dinamico. Qual è il loro potere espressivo?

Dato un blocco qualsiasi se eseguiamo delle operazioni su variabili su ambiente non locale dove ereditiamo queste variabili?

- *Scope statico: ereditarietà dal blocco comune*
- *Scope dinamico: ereditarietà dal blocco precedente a runtime*

Il potere espressivo non cambia, sono entrambi compatibili con la MdT.

Domanda 4

Scope statico implementato con catena statica: come funziona? E scope dinamico con catena dinamica? Vantaggi e svantaggi?

Lo scope statico si implementa attraverso la catena statica: abbiamo uno stack di RdA con 2 puntatori per ogni RdA:

- *puntatore statico (lessicale): punta il RdA dove definiamo il nuovo scope*
- *puntatore dinamico: punta il RdA che ha chiamato*

Se dal RdA k dobbiamo cercare una variabile su ambiente non locale il costo e' $O(n)$.

Lo scope dinamico lo implementiamo con CRT, ovvero la lista dei puntatori dinamici degli RdA (come per la catena statica).

Domanda 5

Come si determina il contatore di catena statica?

Sarebbe la lunghezza massima dei puntatori statici, puo' essere calcolata staticamente guardando quale e' la profondita' massima di annidamento.

Domanda 6

Differenza tra espressione e comando.

- *Espressione: produce un valore ($a+b$)*
- *Comando: puo' non produrre un valore, ma comporta effetti collaterali ($x=5$)*

Domanda 7

Nei linguaggi funzionali ci sono comandi?

No nei linguaggi funzionali non abbiamo comandi, ma solo espressioni.

Domanda 8

Differenza tra iterazione determinata e indeterminata.

L'iterazione determinata e' quando prima di eseguire il ciclo sappiamo il numero di iterazioni (for), mentre quella indeterminata non e' nota a priori (while $x \neq 0$).

Domanda 9

Quali sono i comandi condizionali? Vantaggi tra if e switch.

- *if-else*
- *switch*

Con gli if se abbiamo diversi else if dobbiamo rivalutare ogni volta, mentre con gli switch valutiamo una singola volta e confrontiamo solamente.

Domanda 10

Cos'è il passaggio per riferimento e come funziona?

Si crea la dichiarazione tra la variabile della funzione e l'indirizzo di memoria dell'argomento. Quindi questo viene valutato solo alla chiamata.

Domanda 11

Quando una funzione restituisce un'altra funzione come parametro, cosa succede?

Quando una funzione ritorna un'altra funzione succede che dobbiamo mantenere le variabili anche se lo scope della funzione e' chiuso, perche' se no perderemmo i valori.

Domanda 12

Cos'è la ricorsione in coda?

E' quando una funzione ritorna il valore chiamando se stessa. Puo' essere ottimizzata con la ottimizzazione in coda per utilizzare 1 singolo RdA.

Domanda 13

Cos'è il CRT e a cosa serve?

E' una lista dei puntatori dinamici dei chiamanti, e serve per implementare lo scope dinamico.

3 Modulo 3 – Giallorenzo

Domanda 1

A cosa servono i sistemi di tipi?

Nella programmazione i tipi sono nati:

- *Organizzazione concettuale: definire differenze tra entita'*
- *Astrazione: per legare tipi ad operazioni*
- *Correttezza: type-checking*
- *Implementazione: linguaggio piu' safe*

Domanda 2

Quando un linguaggio è strongly typed e quando è weakly typed?

- *Strongly typed: controlli rigidi sulle operazioni (C, python, ecc)*
- *Weakly typed: supporta piu operazioni ("1" + 5 = "15") (Js)*

Domanda 3

Tipaggio statico e tipaggio dinamico: differenze ed esempi di linguaggi.

- *Statico: controllo statico sui tipi (C)*
- *Dinamico: controllo sui tipi a runtime (python)*

Nel dinamico gli errori arrivano solo quando spunta un controllo non valido.

Domanda 4

Come funziona il sistema di tipi di JavaScript e Python?

Js e python sono a dynamic typing, ma js e' weakly typed mentre python e' strongly typed.

Domanda 5

Controlli statici sui tipi contro controlli dinamici. Cenno all'halting problem.

Il controllo statico si basa sulla deduzione del codice, mentre il dinamico in base alla situazione nel runtime.

Il problema del controllo statico e' che facciamo una deduzione e questo puo' provare situazioni dove blocchiamo anche quando era compatibile. Questo perche' un compilatore che staticamente possa prevedere tutti gli errori ricade nel halting problem.

Domanda 6

Esistono linguaggi non tipati? Che vantaggi ci sono? Dal punto di vista dell'efficienza cambia qualcosa?

Si possono esistere, il vantaggio principale e' la compabilita' assoluta tra tutte le variabili. Ma puo' portare molti bug se il programmatore non sta attento.

Infatti il motivo principale per cui abbiamo creato i tipi e' proprio per rilevare e differire le variabili fra loro.

E' leggermente piu' efficiente dato che non abbiamo controlli.

ASM e' un esempio.

Domanda 7

Differenza tra tipaggio nominale e strutturale. Perché sceglierne uno o l'altro? Pro e contro.

Il tipaggio nominale basa il controllo in base ai tipi delle variabili e la loro compatibilita'. Mentre quello strutturale accetta o no in base alla struttura delle variabili. Il tipaggio strutturale e' piu' comodo perche non bisogna manualmente specificare le relazioni di tipo fra loro e i loro operatori.

Anche l'efficienza puo' cambiare infatti in quello nominale ci basta ci basta confrontare i tag ($O(1)$), mentre in quello strutturale bisogna che cerchiamo dentro la struttura ogni volta.

Quello strutturale pero' potrebbe portare a comportamenti non aspettati se non si sta attenti.

Domanda 8

Cos'è il duck typing e connessione con lo structural typing. Pro e contro.

Con structural typing intendiamo il controllo statico dove una operazione e' compatibile su un tipo a seconda della struttura interna del tipo (e non dal nome), il duck-typing e' lo stesso controllo in python ma a runtime.

Domanda 9

Differenza tra manifest e inferred typing. Pro e contro.

- *Statico: deduciamo la compatibilit  lessicale*
- *Dinamico: controllo a runtime*

Il problema che il controllo statico e' dedotto, non e' assoluto. Infatti dovremmo sapere alcune cose che staticamente sono impossibili. Quindi ci limitiamo a essere conservativi.

Infatti un checker statico perfetto risolverebbe l'halting problem, cosa che per teorema e' impossibile.

Domanda 10

Descrivimi l'algoritmo di unificazione e il suo funzionamento.

E' usato in alcuni linguaggi per dedurre automaticamente i tipi, senza dover avere annotazioni esplicite.

- *Data una qualsiasi funzione determiniamo i tipi in base alle operazioni*
- *Proviamo a sostituire il tipo rilevato nel restante del codice*
- *Rileviamo errori se presenti*

Domanda 11

Che tipaggio usa Java?

Java ha

- *Tipaggio nominale*
- *Tipaggio statico*
- *Manifest (dobbiamo specificare i tipi alle dichiarazioni)*

Domanda 12

Come funzionano equivalenza e comparazione nel typing nominale?

- *Equivalenza: se hanno lo stesso nome*

- *Comparazione: se uno estendo l'altro*

Domanda 13

Cos'è un tipo prodotto? Fai un esempio.

E' un tipo "definito" che si basa su altri tipi "minori"

$Tuple = (int, bool)$

Si chiama prodotto perche il numero di combinazioni possibili e' $|T_1| \times |T_2|$. ($bool \times bool = 4$).

Domanda 14

Cos'è un tipo somma? Qual è la differenza tra tipo somma e tipo prodotto?

Un tipo somma e' un tipo "maggiorato" che si basa sul essere soltanto un tipo singolo definito (enum).

Mentre i tipi prodotto sono una combinazioni che creano un nuovo tipo dato dal prodotto dei tipi minori.

Domanda 15

Differenza tra definizione estensionale e intensionale. Come si combina col sub-typing?

- *Estensionale: definiamo esplicitamente tutti i valori, tipo gli enum ($\{1, 2, 3\}$)*
- *Intensionale: definiamo un insieme ($\{x \mid x > 0\}$)*

Nella definizione intensionale se abbiamo un $A \subset B$ allora $A <: B$.

Mentre in quella estensionale dobbiamo manualmente specificare, come nel typing classico, il sottotipo.

Domanda 16

Sottotipaggio e definizione degli enum. Differenza tra int ed enum.

Dato un tipo T , definiamo sottotipo un suo insieme ristretto di valori chiamato A e lo scriviamo:

$A <: T$

Un enum e un int non puoi calcolarli insieme, mentre due int si. Due enum se non

sottotipati esplicitamente no.

Domanda 17

Definisci un tipo ricorsivo prodotto. Quale potrebbe essere un suo abitante?

Un tipo ricorsivo

$$List(int) = (int, List(int))$$

Il suo abitante e' per esempio (1, (2, (3, ())))).

Domanda 18

Cos'è un tipo ricorsivo?

Un tipo ricorsivo e' un tipo che per definizione include se stesso.

Domanda 19

Come scriverebbe il tipo lista come tipo ricorsivo prodotto?

$$List(T) = (T, List(T))$$

Domanda 20

Cos'è un tipo intersezione?

E' un tipo $T = A \cap B$ che dati due tipi padri A, B e i valori di T coincidono sia dentro A che B .

$$T = A \cap B$$

$$T <: A$$

$$T <: B$$

Ovvero T e' l'insieme dei valori comuni fra A e B .

Domanda 21

Cosa sono i tipi parametrici? In che modo sono utilizzati nel polimorfismo?

I tipi parametrici sono tipi "maggiori" che si implementano basandosi su un'altro tipo

List<T>

Sono utilizzati nel polimorfismo parametrico (universale).

Domanda 22

Quali sono i tipi di polimorfismo?

- *Ad-hoc (sovraccarico/overloading)*
- *Sottotipo*
- *Parametrico (universale)*

Domanda 23

Polimorfismo di sovraccarico e dynamic dispatch: quali sono le differenze?

- *Sovraccarico: e' per le funzioni definite con tipi diversi in I/O*
- *Dynamic-dispatch: la funzione viene chiamata in base al tipo che la ha chiamata*

Domanda 24

Controvarianza e covarianza: quando le troviamo e come mai?

- *Covarianza: restituiamo un sottotipo*
- *Controvarianza: accettiamo un sovratipo*

Servono per ampliare la compabilita fra tipi e funzioni.

Domanda 25

Cos'è il sub-typing?

E' la relazione che ci dice che dove T e' supportato $S <: T$ lo e' ugualmente.

Domanda 26

Nel momento in cui dico che A è un sottotipo di B, quali proprietà deve avere A?

Dato

$$A <: B$$

Sappiamo che tutti i valori di A valgono per B, le operazioni sono definite ugualmente, al massimo abbiamo più valori.

Domanda 27

Principio di sostituzione di Liskov.

E' il principio che ci dice che dato un sottotipo questo e' compatibile ovunque dove utilizziamo il sovratipo.

Domanda 28

Cos'è un sottotipo? Ereditarietà e sottotipi in Java.

Un sottotipo e' un sottinsieme del suo sovratipo. Tutto quello che va con il sovratipo va con il sottotipo.

In Java abbiamo le classi con ereditarietà dove possiamo estendere le classi (ovvero avere sottotipi di classi), e quello che non viene ridefinito lo si eredita' dal padre.

Domanda 29

Quali sono i 4 pilastri dell'OOP? (attenzione: dynamic dispatch al posto di polimorfismo)

- *Incapsulamento: dati-metodi insieme con visibilità (public/protected/private)*
- *Astrazione: solo interfaccia (no metodo)*
- *Ereditarietà*
- *Dynamic dispatch: scelta runtime in base all'oggetto*

Domanda 30

Classi, oggetti, interfacce e prototipi: differenze e caratteristiche.

- *Interfacce: definisce solo i metodi e non implementazione*
- *Prototipi: servono per essere ereditati per avere le implementazioni*
- *Classe: struttura e comportamento degli oggetti*
- *Oggetti: istanza di una classe con valore in memoria con campi*

Domanda 31

Differenza tra sistemi basati su classi e sistemi basati su prototipi. Che garanzie offrono?

- *Classi: oggetto con valore in memoria, funzioni, ecc (statico)*
- *Prototipi: oggetto con alla base delle implementazioni derivanti dal prototipo (puo' variare a runtime)*

Domanda 32

La relazione tra sottotipaggio ed ereditarietà nell'OO. Differenza pratica tra sottotipi e sottoclassi.

*L'ereditarietà nelle classi e' un sottotipo.
Pero' differisce dal sottotipaggio che consiste solo nel avere una versione del padre piu "specifica". La differenza principale e' che il sottotipo e' una versione "specifica" del padre, mentre una sottoclasse puo' ridefinire e ereditarie (dynamic-dispatch).*

Domanda 33

Ereditarietà singola e multipla: differenze e metodi per risolvere i conflitti.

- *Singola: una classe puo' estendere solo un'altra classe*
 - *Multipla: una classe puo' estendere piu' classi, ma questo provoca conflitti.*
- Per risolvere i conflitti nella multipla Java offre le interfacce.*

Domanda 34

Metodi per implementare l'ereditarietà.

Possiamo utilizzare le vtable, ovvero una tabella dove abbiamo tutte le variabili (anche quelle ereditate e sotto-definite), e i puntatori per le funzioni in base alla redefinizioni e tipologia (dynamic-dispatch).

Domanda 35

Perché i costruttori si possono solo sovraccaricare e non fare overriding?

Perche' il costruttore serve per definire la creazione del tipo e non viene ereditato dal padre.

Domanda 36

Differenza tra overriding e overloading.

- *overloading: redefinire una funzione con I/O diverso*
- *overiding: redefinire una funzione in una classe estesa*

Domanda 37

I 2 metodi di implementazione del dynamic dispatch. Qual è il più efficiente?

- *vtable*
- *Message passing: oggetto riceve un messaggio (nome metodo) e lo cerca a run-time*

Vtable piu' efficienti perche' $O(1)$.

Domanda 38

Come funziona l'invocazione dinamica di un metodo? Come avviene l'invocazione Object-oriented?

Attraverso il dynamic-dispatch implementato con le vtable, cerchiamo ed eseguiamo il metodo.

In generale la invocazione nell'OO viene implementata con il MP, quindi e' un msg con il nome del metodo che poi l'oggetto ricerca ed esegue.

Domanda 39

Come si fa in memoria il dynamic dispatch? Come si memorizzano vtable e funzioni?

*Attraverso le vtable (segmento **data** in RAM), una tabella dove memorizziamo puntatori alle variabili (anche quelle sotto-definite) e poi puntatori alle funzioni.*

Domanda 40

In una classe, dove vivono i metodi e i campi e come vengono memorizzati?

Vivono nelle vtable in RAM, in una tabella dove memorizziamo i puntatori alle variabili e puntatori ai metodi.

Domanda 41

Cos'è il problema della classe base fragile?

E' quando estendiamo una classe e sovrascriviamo dei metodi, rischiamo di rompere la logica interna delle funzioni.

Domanda 42

Coercizione e casting: cosa sono e differenze.

- *Casting: traduzione manuale del programmatore (`int x = (int)msg`)*
- *Coercizione: compilatore manualmente traduce perché "sicuro" (`int y; float x = y;`)*

Domanda 43

Differenza tra typed casting e type coercion. Perché non si usano nel polimorfismo generale in Java?

- *Casting: traduzione esplicita del programmatore*
- *Coercizione: traduzione implicita del compilatore*

In java abbiamo il dynamic dispatch, quindi i metodi in base al tipo. Se facciamo un

cast rischiamo che il tipo di sorgente non abbia il metodo

Domanda 44

Differenza tra void e unit.

- *Void: e' l'assenza di un tipo, in una variabile void non puo' esistere e un metodo void non ritorna nulla*
- *Unit: e' un tipo che restituisce il tipo Unit*

Unit serve in quei linguaggi dove le funzioni devono per forza ritornare un valore, quindi con Unit possiamo ritornare il valore Unit

Void invece e' il concetto piu' classico dei LP imperativi dove una funzione puo' svolgere soltanto senza tornare nulla.

Domanda 45

Cos'è il tipo void? Che insieme di valori denota?

Il tipo Void e' un tipo concettuale dei LP imperativi dove un metodo non deve tornare nulla. Il suo insieme non comprende niente.

Domanda 46

Differenza tra maybe e option.

Stesso concetto che serve per specificare per un metodo che ritorniamo un valore o maybe un altro

```
fn foo (int x) -> Option<bool>{  
    if x == 0 return x;  
    else return false;  
}
```

Un esempio in Rust.

Domanda 47

Differenza tra array unidimensionali e multidimensionali. Row-major vs column-major: pro, contro e caching.

Un array unidimensionale e' una lista di valori con un singolo indice. Mentre un array multidimensionale e' una matrice, o piu' con piu' indici.

Possiamo memorizzarli in memoria in ordine:

- *Riga ($A[0][0]$, $A[0][1]$, ...)*
- *Colonna ($A[0][0]$, $A[1][0]$, ...)*

Questo migliora o peggiora le performance (grazie alla cache) in base se e' maggiore il numero di letture per riga o colonna.

Domanda 48

Array in relazione a weakly-typed e C. Vantaggi e svantaggi dell'interoperabilità tra array e puntatori.

In C gli array sono considerati weakly-typed infatti attraverso i puntatori possiamo modificare la memoria come ci pare.

Interoperabilità fra array e puntatori e' interessante perche' ci permette di non fare distinzione in quale segmento di memoria troviamo gli array (stack o heap), e possiamo avere una singola implementazione. Questo pero' comporta a lavorare con i puntatori nei metodi, cosa che ci fa perdere di informazione implicita la lunghezza degli array.

Domanda 49

Come sono memorizzati gli array? Cos'è il dope vector?

Gli array sono memorizzati per

- *Riga ($A[0][0]$, $A[0][1]$, ...)*
- *Colonna ($A[0][0]$, $A[1][0]$, ...)*

Il dope vector e' una tabella con tutte i metadati (informazioni) di un array (Tipo, indirizzo base, ecc).

Domanda 50

Cos'è il dope vector e qual è il suo utilizzo? Dove viene allocato?

Il dope vector e' la tabella di un singolo array con tutti i suoi metadati e lo memorizziamo nello stack.

Domanda 51

Differenza degli array fra Java e C. Come fa Java a gestire gli array?

In Java gli array sono sempre allocati nell'heap. Mentre in C dipende se li allochiamo noi nello stack o heap.

Questo porta che in Java possiamo sempre sapere la lunghezza dell'oggetto, mentre in C nei metodi mai.

Domanda 52

Garbage collection: quali implementazioni abbiamo visto e come funzionano?

- *RC*
- *Stop and copy*
- *Mark and sweep*
- *Borrow checking*

I primi 3 sono a runtime, mentre il 4 e' statico.

Domanda 53

Garbage collection: come funzionano i contatori dei riferimenti?

Per ogni oggetto nel heap manteniamo un contatore che ci indica quanti sono i puntatori che ne fanno riferimento, quando arriviamo a 0 deallochiamo.

Domanda 54

Limiti del reference counting.

I cicli, due oggetti che puntano a se stessi non verranno mai dealloati. Poi anche il costo in memoria di tutti i contatori, del costo di controllo se dobbiamo deallocare e che la deallocazione viene appena un oggetto e' trash.

Domanda 55

Garbage collector con il costo più basso di memoria?

Borrow checking perche' essendo statico non implica overhead.

Domanda 56

Problemi di un garbage collector stop-the-world. In quali applicazioni crea problemi? Alternative?

E' un problema nei sistemi realtime con magari IO, infatti un blocco del programma per spostare e ripulire la memoria rallenta e peggiora notevolmente le performance. Una alternativa o RC se il costo di overhead e' accettabile o Borrow checking.

Domanda 57

Cosa significa che un valore non è garbage?

Un valore non e' garbage quando quel valore e' ancora raggiungibile attraverso un quale oggetto.

Domanda 58

Cosa sono i dangling pointers e le dangling reference?

Un dandling pointer/reference e' un puntatore che punta a una cella di memoria liberata.

Domanda 59

Cos'è una use after free?

E' quando cerchiamo di dereferenziare un puntatore che puntava a una cella che e' stata liberata, questo porta a un segfault.

Domanda 60

Cosa sono i wide pointers / initialized pointers?

- *Wide pointers: puntatori che hanno info aggiuntive (come len di un array)*
- *Initialized pointers: puntatori che alla creazione non punta una cella a caso (Rust)*

Domanda 61

Tecniche di protezione della memoria e loro problemi.

- *Garbage Collector*
- *Wide pointers*
- *Non avere puntatori (e gestione della memoria direttamente)*

Domanda 62

Cosa sono le eccezioni? A cosa servono? Come sono gestite in Java?

Tecnica nei LP per lanciare un segnale per interrompere un metodo e ed eseguire un pezzo di codice specifico quando un gestore compatibile la rileva.

Servono per ampliare le opzioni di programmazione in un linguaggio e semplificare la vita al programmatore.

In Java abbiamo Errori e Eccezioni che entrambe possono essere lanciate, alcune vanno gestite altre no.

Domanda 63

Ci sono similitudini tra eccezioni e tipaggio?

Si le eccezioni catturano proprio per tipo, infatti ci basiamo su polimorfismo di sottotipo.

Domanda 64

Tipi di gestione degli errori. Cos'è il result type? Cos'è il tipo option/-maybe?

- *Classico: ritorniamo un valore specifico tipo EXIT_ERROR*
- *Result Type: ritorniamo un tipo enum dove abbiamo il risultato o un errore*

```
enum R {  
    result;  
    error;  
}
```

Domanda 65

Differenza tra abstract data types e oggetti. Possono convivere nello stesso codice?

- *ADT: un tipo ha già delle operazioni integrate, e il programmatore non se ne deve preoccupare*
- *Oggetto: definiamo noi i tipi ma dobbiamo creare tutte le implementazioni*

*Se vogliamo aggiungere nuovi tipi gli oggetti sono più comodi, mentre se vogliamo aggiungere nuove funzioni possiamo utilizzare gli ADT.
In Java abbiamo entrambi anche contemporaneamente.*

Domanda 66

Definisci i tipi di dato astratti.

*Un ADT è un tipo che ha già dei metodi implementati internamente, il programmatore non se ne deve preoccupare.
Sono comodi per creare nuove funzioni, ma scomodi per creare nuovi tipi (dovremo modificare TUTTI i metodi interni per renderlo compatibile con il nuovo).*

Domanda 67

Cosa vuol dire dal punto di vista concettuale e pratico avere diverse visibilità (pubblica/privata)?

- *Concettuale: separiamo interfaccia dalle implementazioni*
- *Pratico: accessibile sono all'interno della classe*

Domanda 68

Differenza tra parte pubblica e parte privata.

*Pubblica tutti possono accedervi anche da fuori dalla classe, privata sono internamente alla classe possiamo accedervi.
Questo si collega con anche la logica interfaccie-implementazione.*

Domanda 69

Se le funzioni fossero come le funzioni matematiche, come potremmo ottimizzarne l'esecuzione?

Potremo trasformalo in funzioni deterministiche così da cachare il risultato se ci riserve.

Domanda 70

Cos'è una relazione? Come ottimizzare il calcolo di una funzione con dominio molto piccolo?

*Una relazione e' l'insieme prodotto di due tipi.
Se abbiamo un dominio molto piccolo potremmo precalcolare tutti i risultati per poi accederci in $O(1)$.*

Domanda 71

Posso implementare la comunicazione sincrona con quella asincrona?

Si certamente, mentre il contrario no (senza un processo esterno server).